

[Open in app](#) Search Write

Monte Carlo Simulation on Stock Price Forecasting Using Python



Simon Leung · 7 min read · Oct 18, 2024



Image from investopedia.com

What is Monte Carlo Simulation?

Monte Carlo Simulation is a sophisticated computational technique that employs iterative random sampling based on predefined probability distributions (e.g., normal, uniform) to generate numerical outcomes. This method constructs models of potential results by substituting a spectrum of values — represented by a probability distribution — for any variable exhibiting inherent uncertainty. The simulation then performs repeated calculations, utilizing a distinct set of random values from the probability functions in each iteration.

Originally developed to address complex physical and mathematical challenges, the Monte Carlo method has since found widespread application across diverse domains, including but not limited to:

1. Financial modeling and risk assessment
2. Engineering and systems reliability analysis
3. Supply chain optimization
4. Project management and scheduling
5. Environmental and climate modeling
6. Operations research and decision analysis

The versatility and robustness of Monte Carlo Simulation make it an invaluable tool for quantifying uncertainty and facilitating data-driven decision-making in scenarios characterized by multiple interdependent variables and stochastic processes.

Monte Carlo simulation process:

1. **Model Definition:** Establish the parameters and equations that describe your system or process.
2. **Random Input Generation:** Create random inputs based on predetermined probability distributions.
3. **Simulation Execution:** Run the model multiple times, each time using a different set of random inputs.
4. **Results Analysis:** Examine the outcomes from numerous simulations to gain insights into the system's behavior.

Disclaimer:

The information presented herein is provided solely for informational purposes and should not be construed as financial or investment advice. Any investment decisions or actions taken based on this information are made at your own discretion and risk. Stock prices and investment values are subject to fluctuation due to various market factors and conditions.

I strongly recommend consulting with a qualified financial professional for personalized guidance tailored to your individual financial situation and objectives before making any investment decisions.

Let's start it with one of my Roth IRA mutual fund, T. Rowe Price Equity Index 500 Fund (PREIX), as an example:

[\[Full source code here\]](#)

Step 1: Gather Historical Data

```
# Helper Function to fetch historical stock data
import yfinance as yf
def get_stock_data(ticker, start_date, end_date):
    try:
        data = yf.download(ticker, start=start_date, end=end_date)
        return data
    except Exception as e:
        return f"Error fetching data: {e}"

ticker_symbol = "PREIX" # T. Rowe Price Equity Index 500 Fund
start_date = "2000-01-01"
end_date = "2024-10-01"
stock_data = get_stock_data(ticker_symbol, start_date, end_date)
stock_data.asfreq('B').index # set index frequency to business daily
```

Check the “Close” price Stationarity — A time series is nonstationary if its statistical properties change over time.

```
# Augmented Dickey-Fuller test for stationarity
result = adfuller(stock_data['Close'], regression='ctt')
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))
if result[1] > 0.05 :
    print('Fail to reject the null hypothesis (H0), the data has a unit root and')
elif result[1] <= 0.05 :
    print('Reject the null hypothesis (H0), the data does not have a unit root a')
```

```
ADF Statistic: -3.051038
p-value: 0.271434
Critical Values:
    1%: -4.373
    5%: -3.833
```

10%: -3.554

Fail to reject the null hypothesis (H_0), the data has a unit root and is non-sta

We calculate the daily returns based on the percentage of the close price changes. Daily Return= (Price Today–Price Yesterday) / Price Yesterday.[1]

```
# calculate the daily returns of the stock investment based on the percentage of
# The pct_change method will automatically calculate the percentage changes of t
stock_data['Daily_Return'] = stock_data['Close'].pct_change()
stock_data.dropna(inplace=True)
```

Check 'Daily_Return' Stationarity:

```
ADF Statistic: -15.150224
p-value: 0.000000
Critical Values:
1%: -4.373
5%: -3.833
10%: -3.554
Reject the null hypothesis ( $H_0$ ), the data does not have a unit root and is stati
```

A stationary time series is one whose statistical properties such as mean, variance, autocorrelation, etc. are all constant over time.

Step 2: Calculate Mean and Standard Deviation

Using Goodness-Of-Fit test to see Normal (Gaussian) Distribution or Student's t-Distribution fits the 'Daily_Return' data

The Anderson-Darling test — for normal distribution

```
from scipy.stats import anderson

# Perform Anderson-Darling test
result = anderson(stock_data['Daily_Return'], dist='norm')
print('Statistic: %.3f' % result.statistic)

for i in range(len(result.critical_values)):
    sl, cv = result.significance_level[i], result.critical_values[i]
    print('Significance Level: %.3f, Critical Value: %.3f' % (sl, cv))
    if result.statistic < cv:
        print('Sample looks normal (fail to reject H0)')
    else:
        print('Sample does not look normal (reject H0)')
```

```
Statistic: 115.617
Significance Level: 15.000, Critical Value: 0.576
Sample does not look normal (reject H0)
Significance Level: 10.000, Critical Value: 0.656
Sample does not look normal (reject H0)
Significance Level: 5.000, Critical Value: 0.786
Sample does not look normal (reject H0)
Significance Level: 2.500, Critical Value: 0.917
Sample does not look normal (reject H0)
Significance Level: 1.000, Critical Value: 1.091
Sample does not look normal (reject H0)
```

The Kolmogorov-Smirnov test — for Student's t-Distribution

```

from scipy.stats import kstest, t

# Fit the data using t-distribution
params = t.fit(stock_data['Daily_Return'])
d, p_value = kstest(stock_data['Daily_Return'], 't', args=params)
print(f"KS Statistic: {d}, P-Value: {p_value}")

if p_value > 0.05:
    print('Sample fits the t-distribution (fail to reject H0)')
else:
    print('Sample does not fit the t-distribution (reject H0)')

```

KS Statistic: 0.01693130433264617, P-Value: 0.05579082151128423
Sample fits the t-distribution (fail to reject H0)

From the test, “Daily_Return” fits the Student’s t-Distribution

```

from scipy.stats import t

# Fit a t-distribution to the daily return data
params = t.fit(stock_data['Daily_Return'])

# Get the parameters
dfree, mu, std = params
print(f"Degrees of Freedom: {dfree}, Mean: {mu}, Standard Deviation: {std}")

# Plot the histogram and the PDF
plt.hist(stock_data['Daily_Return'], bins=50, density=True, alpha=0.6, color='g')

xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = t.pdf(x, dfree, mu, std)

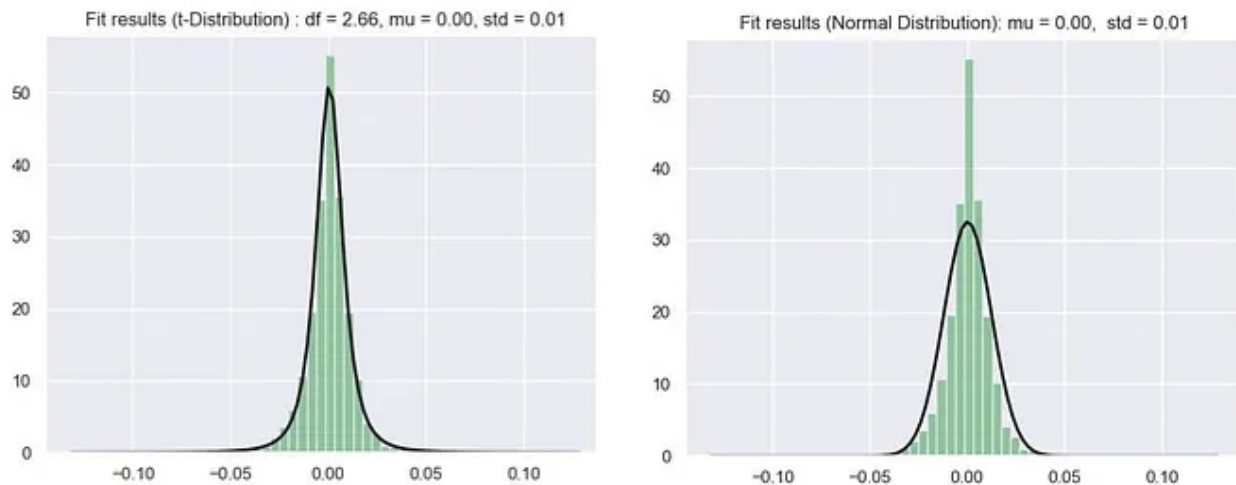
plt.plot(x, p, 'k', linewidth=2)
title = f"Fit results (t-Distribution) : df = {dfree:.2f}, mu = {mu:.2f}, std = {std:.2f}"
plt.title(title)

plt.show()

```

```
# Save the figure
#plt.savefig('t_dist.jpg')
```

Degrees of Freedom: 2.6606212055936953, Mean: 0.0006423654065788996, Standard De



Step 3: Simulate Future Returns based on Student's t-Distribution.
Simulate stock price paths by:

$$P_{t+1} = P_t \cdot (1 + \text{return}\%)$$

Image from [2]

```
import scipy.stats as stats
import numpy as np
import plotly.graph_objects as go

# Fit a t-distribution to the daily return data
params = stats.t.fit(stock_data['Daily_Return'])
df_t, loc, scale = params[0], params[1], params[2]
```



```

# Number of simulations and forecast days
num_simulations = 1000
forecast_days = 252 # 1 year of trading days

# Generate random samples using t-distribution
simulated_returns = stats.t.rvs(df_t, loc=loc, scale=scale, size=(forecast_days,

# Starting stock price
start_price = stock_data['Close'].iloc[-1]

# Simulate stock price paths
price_paths = np.zeros_like(simulated_returns)
price_paths[0] = start_price

for t in range(1, forecast_days):
    price_paths[t] = np.maximum(price_paths[t - 1] * (1 + simulated_returns[t]),

# Calculate the 5th and 95th percentiles of the stock prices
lower_bound = np.percentile(price_paths, 5, axis=1)
upper_bound = np.percentile(price_paths, 95, axis=1)

# Create a figure
fig = go.Figure()

# Add traces for each simulation
for i in range(num_simulations):
    fig.add_trace(go.Scatter(y=price_paths[:, i], mode='lines', line=dict(color=

# Add traces for the lower and upper bounds
fig.add_trace(go.Scatter(y=lower_bound, mode='lines', name='5th Percentile', li
fig.add_trace(go.Scatter(y=upper_bound, mode='lines', name='95th Percentile', li

# Update layout
fig.update_layout(title='Monte Carlo Simulation of Stock Prices with 90% Confide
                    xaxis_title='Days',
                    yaxis_title='Stock Close Price $', width=1000, height=600)

# Show the figure
fig.show()

# Save the figure as a JPG image
fig.write_image("MonteCarloSim.jpg")

```



From the simulation, the “Close” price starts at USD151.56 (2024–09–30), the price can be as high as ~USD250 or as low as ~USD100 within 90% confidence interval for next 252 days (1 year).

Monte Carlo methods are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. Here’s a breakdown of their strengths and weaknesses:

Strengths:

1. Versatility: Can be applied to a wide range of problems in various fields.

2. Handling complexity: Effective for solving problems with many coupled degrees of freedom.
3. Probabilistic insights: Provides probabilistic results, useful for risk analysis.
4. Dimensionality: Can handle high-dimensional problems well.
5. Parallelization: Easily parallelizable, making them suitable for modern computing architectures.
6. Non-linear systems: Can handle non-linear systems and complex geometries.
7. Intuitive approach: Often more intuitive than analytical methods for complex systems.
8. Flexibility: Can incorporate different probability distributions easily.
9. Continuous improvement: Results improve with increased number of simulations.
10. Model testing: Useful for validating and testing complex models.

Weaknesses:

1. Computational intensity: Can be computationally expensive, especially for high-precision results.

2. Precision vs. time trade-off: Achieving high precision often requires long computation times.
3. Pseudorandom number quality: Results can be affected by the quality of random number generators.
4. Rare event simulation: May struggle with simulating rare events efficiently.
5. Deterministic problems: Not always the best choice for purely deterministic problems.
6. Interpretation challenges: Results can be difficult to interpret without statistical expertise.
7. Convergence issues: May face slow convergence for certain types of problems.
8. Model dependency: The quality of results depends heavily on the quality of the underlying model.
9. Sensitivity to initial conditions: Results can be sensitive to initial parameters and distributions.
10. Verification difficulty: It can be challenging to verify that the implementation is correct.

Additional articles on this topic:

[Monte-Carlo simulation for stock Price Forecasting: Python guide.](#)

[How to Simulate the Stock and Cryptocurrency Price Data Using Monte Carlo Simulation.](#)

[Random Stock Generator — Monte Carlo Simulations in Finance.](#)

[A practical guide to using Monte Carlo simulations for predicting stock prices.](#)

[Monte Carlo Simulations for Stock Price Predictions \[Python\].](#)

[198. Predicting the Unpredictable: Analyzing Potential Stock Prices with Monte Carlo Simulations.](#)

[Predicting Stock Prices Using Monte Carlo Methods in Python.](#)

[Predicting Stock Prices with Monte Carlo Simulations.](#)

[Python and Finance: Stock Prediction Using Monte Carlo Best Fit Method.](#)

Recommended Books:

[Monte Carlo with Python](#), Van Der Post, Hayden, (2024, Reactive Publishing)

[Financial Data Analytics with R: Monte-Carlo Validation](#), Jenny Chen (2024, CRC Press)

Reference:

[1] [Analyzing Stock Returns and Volatility with Python](#)

[2] [Monte Carlo Simulation for Time Series Probabilistic Forecasting](#)

Thank you for reading.



Written by Simon Leung

229 followers · 4 following

Edit profile

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.

